

RESEARCH ARTICLE

Building an intrusion detection system on UNSW-NB15: Reducing the margin of error to deal with data overlap and imbalance

Zeinab Zoghi¹ | Gursel Serpen

Electrical Engineering & Computer Science,
University of Toledo, Toledo, Ohio, USA

Correspondence

Zeinab Zoghi, Electrical Engineering &
Computer Science, University of Toledo,
Toledo, OH 43606, USA.
Email: zzoghi@rockets.utoledo.edu

Summary

This study addresses the challenge of data imbalance and class overlap in machine learning for intrusion detection, proposing that targeted algorithmic adjustments can significantly enhance model performance. Our hypothesis contends that an ensemble framework, adeptly integrating novel threshold-adjustment algorithms, can improve classification sensitivity and specificity. To test this, we developed an ensemble model comprising Balanced Bagging (BB), eXtreme Gradient Boosting (XGBoost), and Random Forest (RF), fine-tuned using grid search for BB and XGBoost, and augmented with the Hellinger metric for RF to tackle data imbalance. The innovation lies in our algorithms, which adeptly adjust the discrimination threshold to rectify the class overlap problem, enhancing the model's ability to discern between negative and positive classes. Utilizing the UNSW-NB15 dataset, we conducted a comparative analysis for binary and multi-category classification. Our ensemble model achieved a binary classification accuracy of 97.80%, with a sensitivity rate of 98.26% for detecting attacks, and a multi-category classification accuracy and sensitivity that reached up to 99.73% and 97.24% for certain attack types. These results substantially surpass those of existing models on the same dataset, affirming our model's superiority in dealing with complex data distributions prevalent in network security domains.

KEYWORDS

anomaly detection, binary classification, class imbalance, class overlap, cybersecurity analytics, ensemble learning, intrusion detection systems (IDS), machine learning, multi-class classification, network security, UNSW-NB15 dataset

1 | INTRODUCTION

Developing an effective intrusion detection system (IDS) through machine learning approaches necessitates a thorough analysis of datasets to identify and address inherent dataset issues for optimal classifier performance.¹ Our preceding study² analyzed the UNSW-NB15 dataset, revealing significant challenges of class imbalance^{3,4} and class overlap,⁵ including both between-class and within-class imbalances. This paper proposes an ensemble classifier⁶ that integrates Balanced Bagging (BB),^{7,8} eXtreme Gradient Boosting (XGBoost),⁹ and Random Forest (RF)¹⁰ with Hellinger Distance Decision Trees (RF-HDDT) to address these issues. Tree-based algorithms have shown to be effective for the UNSW-NB15 dataset.^{11–21} We have fine-tuned the hyper-parameters of BB and XGBoost, and utilized the Hellinger distance^{22,23} in RF to measure class similarities in unevenly

This is an open access article under the terms of the [Creative Commons Attribution-NonCommercial-NoDerivs](https://creativecommons.org/licenses/by-nc-nd/4.0/) License, which permits use and distribution in any medium, provided the original work is properly cited, the use is non-commercial and no modifications or adaptations are made.

© 2024 The Author(s). *Concurrency and Computation: Practice and Experience* published by John Wiley & Sons Ltd.

distributed data. Moreover, we introduce two novel algorithms to set the discrimination threshold, thereby improving the model's performance in the presence of class overlap.

The literature on machine learning approaches in the context of intrusion detection is extensive.²⁴ has proposed an algorithm combining invasive weed optimization with a differential evolutionary model to optimize perceptron neural network coefficients for medical diagnosis accuracy. Alzubi et al. introduced a dynamic programming-based ensemble design algorithm (DPED), improving accuracy and reducing ensemble size, validated across various UCI repository datasets.²⁵ An innovative approach has been presented for Android malware detection, combining SVM classifiers with the Harris Hawks Optimization (HHO) algorithm for feature weighting and hyperparameter optimization.²⁶ Furthermore, a technique known as FDL-CADIS utilizes machine learning and deep learning models to detect and classify cyberattacks and malware, transforming API call sequences into images for classification with deep learning models, outperforming traditional classifiers on the Maling dataset.²⁷

Taking these related works into account, our proposed model is designed to tackle the specific challenges posed by the UNSW-NB15 dataset. The preprocessing phase includes data cleaning, scaling, and feature selection, followed by training, validation, and testing of our ensemble model. The novel algorithms we propose, are applied to adjust classification thresholds based on prediction errors, improving the model's sensitivity to class overlaps. The forthcoming sections will detail the UNSW-NB15 dataset, preprocessing steps, algorithmic contributions, and a comprehensive evaluation of our model against state-of-the-art IDS frameworks.

2 | DATASET

The UNSW-NB15 network security dataset was released in 2015 by Moustafa et al.^{28,29} This dataset comprises 2,540,044 records, available through the UNSW webpage³⁰ and contains realistic, modern normal and malicious network activities. The structure of this dataset is more complex compared to other benchmark datasets,^{31–35} making UNSW-NB15 more comprehensive and reliable for evaluating network intrusion detection systems.³²

The records for the UNSW-NB15 dataset were gathered by the IXIA traffic generator³⁶ using three virtual servers. Two servers were configured to distribute the normal traffic packets and the third one was configured to spread abnormal traffic packets. A total of 49 features, including packet-based and flow-based ones, were extracted from the records using Argus³⁷ and Bro-IDS³⁸ tools. Packet-based features are extracted from a packet header and its payload (also called packet data). In contrast, flow-based features are produced from the sequence of packets traveling between a source and a destination. The features are categorized into basic, content and time categories, while others are labeled as general-purpose features and connection features. General-purpose features explain the purpose of an individual record, while connection features capture the characteristics of the connection between a 100 sequentially-ordered records. The last two features represent attack categories and labels.

Attacks are categorized as Analysis, Backdoor, DoS, Exploits, Fuzzers, Generic, Reconnaissance, Shellcode, and Worms. The Normal class contains 2,218,761 records, while Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms signatures include 24,246, 2677, 2329, 16,535, 44,525, 215,481, 13,987, 1511, and 174 records, respectively. The dataset is subsampled and split into training and testing subsets. The training split contains 175,341 records and the test split contains 82,332 records, each of which contains the records belonging to 9 different attack classes, as detailed in Table 1.

TABLE 1 Number of records in training and testing subsets for each class.

Classes	Training subset	Testing subset	All
Normal	56,000	37,000	93,000
Analysis	2000	677	2677
Backdoor	1746	583	2329
DoS	12,264	4089	16,353
Exploits	33,393	11,132	44,525
Fuzzers	18,184	6062	24,246
Generic	40,000	18,871	58,871
Reconnaissance	10,491	3496	13,987
Shellcode	1133	378	1511
Worms	130	44	174
Total Number of Records	175,341	82,332	257,673

The class distribution in UNSW-NB15 is shown in Figure 1. The skewed class proportion represents the between-class imbalance. In between-class imbalance, although the distribution of the sub-classes within minority classes is even, as is within majority classes, the sample size between the majority (Normal) and minority (Worms) classes differs.

To measure the severity of the class imbalance, the Imbalance Ratio (IR: majority class distribution/minority class distribution) is calculated for the UNSW-NB15 samples. Since UNSW-NB15 is a multi-class dataset, it is decomposed into $m(m-1)/2$ binary-class subsets using the One-Versus-One (OVO) method, where m is the number of classes. Then, the IR is computed for each class against the rest. According to Fernández, Alberto, et al.,³⁹ any IR value greater than 1.5 is considered imbalanced.

The occurrence of the classes, empirical class distribution, and the IR is shown in Table 2. The greatest and smallest IR values are 514.29 and 1.50, respectively, indicating that Normal and Worms classes possess the most, while Fuzzers and DoS have the least imbalance issues. Since the

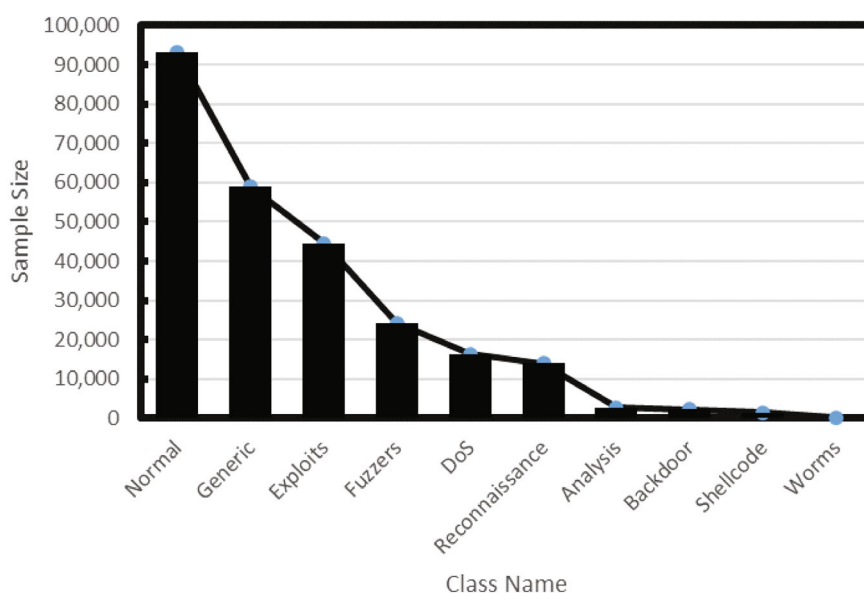


FIGURE 1 UNSW-NB15 class distribution.

TABLE 2 Empirical class distribution and imbalance ratios.

Occurrence						Empirical class distribution				
93,000/2677/2329/16,353/44,525/24,246/58,871/13,987/1511/174						0.36/0.01/0.01/0.06/0.17/0.09/0.23/0.05/0.01/0.0007				
IR										
Class name	Normal	Analysis	Backdoor	DoS	Exploits	Fuzzers	Generic	Recon	Shellcode	Worms
Normal	1	36.00	36.00	6.00	2.12	4.00	1.57	7.20	36.00	514.29
Analysis		1	1	6.00	17.00	9.00	23.00	5	1	14.29
Backdoor			1	6	17	9	23	5	1	14.29
DoS				1	2.83	1.50	3.83	1.2	6	85.71
Exploits					1	1.89	1.35	3.4	17	242.86
Fuzzers						1	2.56	1.80	9.00	128.57
Generic							1	4.60	23	328.57
Reconnaissance								1	5	71.43
Shellcode									1	14.29
Worms										1

Note: The shading indicates significant imbalance ratios greater than 1.5.

imbalance threshold is 1.5, there is no imbalance issue between Backdoor/Analysis, Generic/Exploits, Reconnaissance/DoS, Shellcode/Analysis, and Shellcode/Backdoor.

The high IR values in Table 2 between most of the classes in UNSW-NB15 indicate that the imbalance issue in this dataset is severe and needs to be addressed. According to Ref. 2 there is also an overlapping issue in this dataset. Thus, the class imbalance issue becomes more serious when class overlap is also present in the dataset, which is considered in this study.

3 | DATA PREPROCESSING AND EXPLORATORY CLASSIFIER EVALUATION

In this section, several classifiers are evaluated, and those with high performance are selected to form the proposed model. Once the top classifiers were chosen, a multistep preprocessing approach⁴⁰ was implemented on UNSW-NB15 to prepare the data for effective model training. Before preprocessing, source IP address, source port number, destination IP address, destination port number, record_start_time, and record_last_time features were removed, as these features potentially did not contribute to developing a classifier with general applicability.^{41–45}

SVM, Naïve Bayes (NB), Bagging, Multi-Layer Perceptron Neural Network (MLP-NN), Random Forest (RF), Extremely Randomized Trees (ERT), AdaBoost, Gradient Tree (GT), Balanced Bagging (BB), XGBoost, and Easy Ensemble (EE) were employed on the UNSW-NB15 dataset with the remaining 42 features to identify high-performing base classifiers for an ensemble design. The results, in terms of missed alarm rate, are shown in Table 3.

Table 3 reveals that BB, XGBoost, and RF lead in their performance. The top classifiers were implemented along with various scalers such as Normalizer, Min-Max scaler, Robust scaler, Standard scaler, Quantile transformer, and Power transformer to identify which scalers would enhance classifier performance. The results, presented in Table 4 in terms of accuracy, show that the Min-Max Scaler improved classifier performance. The

TABLE 3 Missed alarm rate values for the set of 11 classifiers.

Classes	SVM	NB	Bagging	MLP-NN	RF	ERT	AdaBoost	GT	BB	XGBoost	EE
Analysis	1.00	1.00	0.99	1.00	0.86	1.00	0.87	1.00	0.77	0.87	0.86
Backdoor	1.00	1.00	0.93	0.98	0.76	0.95	0.93	0.95	0.59	0.64	0.60
DoS	0.90	0.99	0.88	0.98	0.87	0.87	0.99	0.93	0.81	0.83	0.99
Exploits	0.90	0.97	0.21	0.82	0.21	0.28	0.70	0.09	0.42	0.04	0.99
Fuzzers	0.94	0.63	0.42	0.89	0.39	0.42	0.93	0.55	0.32	0.29	0.75
Generic	0.51	0.03	0.03	0.03	0.03	0.03	0.42	0.03	0.04	0.02	0.42
Normal	0.04	0.65	0.24	0.24	0.21	0.23	0.86	0.34	0.34	0.39	0.70
Recon	0.65	0.71	0.19	1.00	0.17	0.22	0.17	0.21	0.17	0.18	0.99
Shellcode	0.96	0.98	0.31	1.00	0.30	0.52	0.92	0.60	0.06	0.12	0.81
Worms	1.00	0.95	0.86	1.00	0.04	0.86	1.00	0.56	0.09	0.43	0.88

Note: The highlighted columns represent RF, BB, and XGBoost which demonstrated superior performance compared to other classifiers.

TABLE 4 Accuracy comparison among the performances of six different scalers.

	BB (%)	XGBoost (%)	RF (%)
Original Dataset	87.56	94.11	92.81
Normalizer	81.45	85.02	85.78
Min-Max Scaler	88.91	96.40	93.33
Robust Scaler	87.61	94.55	93.12
Standard Scaler	86.64	92.73	91.99
Quantile Transformer	87.32	93.95	93.12
Power Transformer	86.11	92.00	91.27

Note: The highlighted row represents the improved accuracy scores for BB, XGBoost, and RF when the Min-Max scaler is applied.

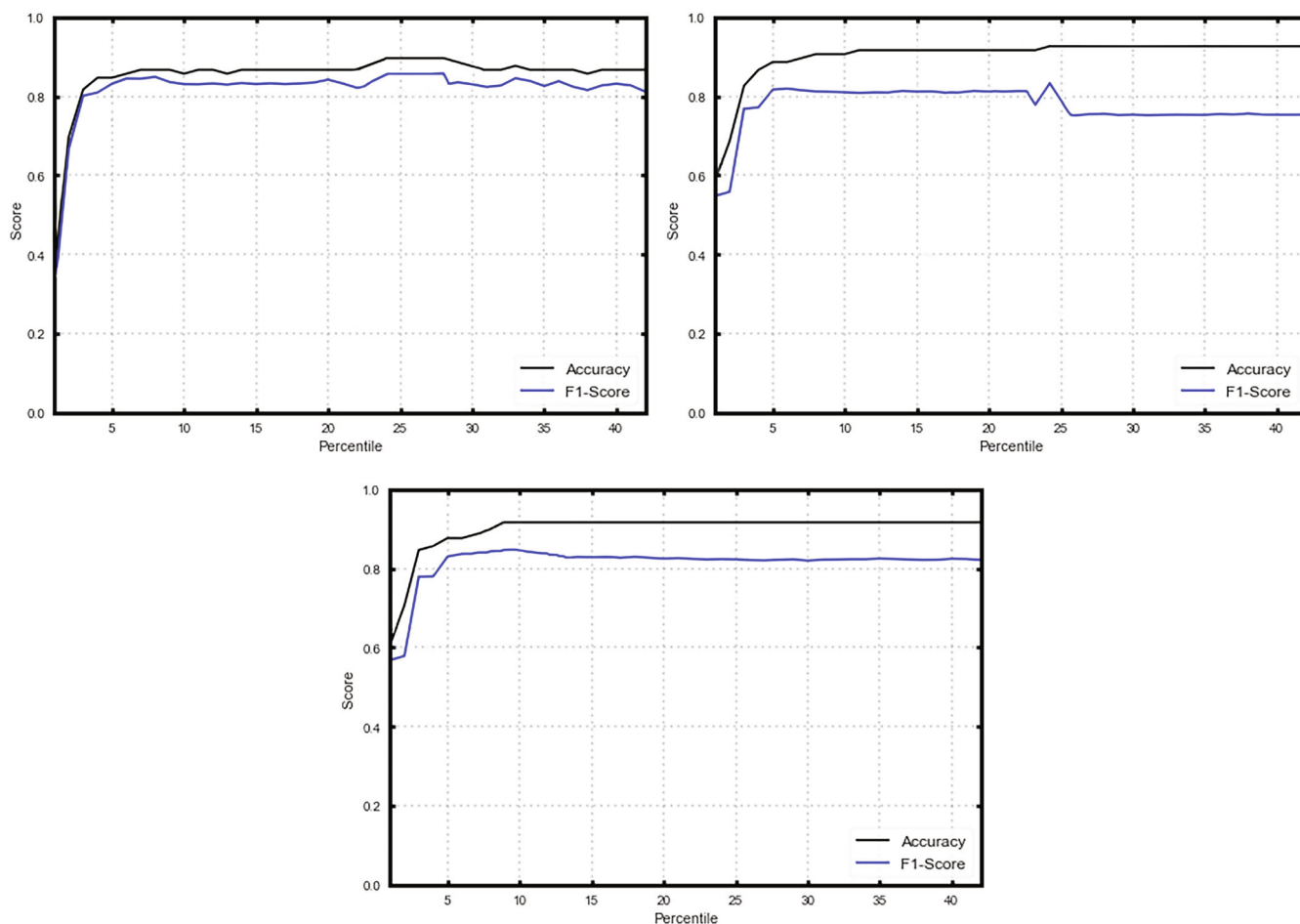


FIGURE 2 The accuracy and F1-score of BB (left), XGBoost (center), and RF(right) classifiers.

first row represents accuracy scores where the classifiers were implemented on the original dataset, while the remaining row show accuracy scores with scaled dataset. Comparing the first line with the rest indicates that the Min-Max scaler improved classifier performance.

In the final preprocessing step, the most relevant subset of features was selected from the remaining 42. They were identified by repeatedly implementing the high-performing classifiers on UNSW-NB15, along with seven feature selection algorithms: Chi-Squared, Genetic Algorithm, mRMR, Relief, Recursive Feature Elimination (RFE), Sequential Feature Selection (SFS), and Elastic Net. In each iteration, one feature was added to the previous set of features selected in earlier iterations. The performance of the classifiers was evaluated in every iteration using Accuracy and F1-score to monitor True Positive/Negative and False Positive/Negative rates. BB and XGBoost classifiers delivered better accuracy and F1-score with 24 out of 42 features selected by Elastic Net while RF performed better with 8 features selected by SFS from the 24 features initially selected for BB and XGBoost. Figure 2 shows how well the classifiers performed with different number of features.

The best results from the top three classifiers were selected for Figure 2. Feature selection was implemented alongside the top three classifiers. BB and XGBoost performed as well with 24 features selected by Elastic Net as with 42 features, but RF performed best with 8 features selected by SFS. Since the 8 features were among the 24 selected for BB and XGBoost, the final design was developed from the hierarchy of Elastic Net and SFS. The top features that improved the performance of the classifiers are shown in Table 5.

Min-max scaling was implemented on the selected features prior to the training and testing phases in the proposed model. The preprocessing design for the test phase is similar to that of the training phase.

4 | CLASSIFIER DEVELOPMENT METHODOLOGY

To address the imbalanced data, we utilized the BB and the XGBoost ensemble classifiers which offer a set of hyper-parameters that, when judiciously adjusted, contribute to improved performance in the presence of imbalanced data. To tackle the class overlap problem, we proposed two new algorithms, namely, Algorithms #1 and #2, presented in sections 4.1 and 4.2, respectively. These algorithms process the classification outputs from

TABLE 5 List of 24 features selected by Elastic Net and 8 features selected by SFS.

Feature Name	Elastic Net	SFS	Description
dur	X		Record total duration
proto	X	X	Transaction protocol
service	X	X	Contains the network services
state	X		Contains the state and its dependent protocol
spkts	X		Source to destination packet count
dpkts	X		Destination to source packet count
sbytes	X	X	Source to destination transaction bytes
dbytes	X		Destination to source transaction bytes
rate	X	X	Ethernet data rates transmitted and received
sttl	X		Source to destination time to live value
dttl	X		Destination to source time to live value
sload	X		Source bits per second
dload	X	X	Destination bits per second
sloss	X		Source packets retransmitted or dropped
dloss	X		Destination packets retransmitted or dropped
sinpkt	X		Source interpacket arrival time (msec)
dinpkt	X		Destination interpacket arrival time (msec)
sjit	X	X	Source jitter (msec)
djit	X	X	Destination jitter (msec)
swin	X		Source TCP window advertisement value
stcpb	X		Destination TCP window advertisement value
dtcpb	X		Destination TCP base sequence number
dwin	X		Destination TCP window advertisement value
tcprrt	X	X	TCP connection setup round-trip time

the BB and XGBoost classifiers. During the training phase, the algorithms calculate the predicting errors, the difference between the expected predicting scores and the achieved predicting scores, resulting from the misclassification of one or multiple classes. During the test phase, the decision scores are revised to adjust the sensitivity of the machine learning models to the outcome based on the calculated predicting errors.

Furthermore, the performance of RF classifier on the UNSW-NB15 dataset was notably weak, particularly in terms of missed alarm rate. Decision Trees, the basis of RF, suffer from several issues such as overfitting, axis-parallel splitting, and skewness sensitivity. The overfitting problem is mitigated through tree pruning, and axis-parallel splitting can be addressed by constructing a forest of orthogonal¹⁰ and oblique decision trees.⁴⁶ Additionally, the skewness sensitivity of decision trees, often exacerbated by popular splitting criteria like information gain and the Gini measure. Hellinger's measure can address this problem due to its skew insensitivity property. Therefore, Hellinger distance is leveraged as the split criterion in the construction of trees rather than the default entropy value^{23,47} used in RF. After tuning the hyper-parameters in BB and XGBoost and addressing skewness sensitivity in RF, the outputs of the classifiers were provided as inputs to a combiner that implemented majority voting to determine the final class membership of a data record under test.

4.1 | Training

The training dataset is divided into two subsets: a training subset and a validation subset, using stratified random sampling⁴⁸: 80% of the training set is used to train the proposed model and the remaining 20% is employed to calculate errors arising from incorrect predictions by the BB and XGBoost classifiers. These error calculations are performed by Algorithm #1, which will be detailed in subsequent paragraphs.

TABLE 6 First seven rows of a probability score matrix generated by BB classifier.

Row index	Analysis	Backdoor	DoS	Exploit	Fuzzers	Generic	Normal	Recon	Shellcode	Worms
0	0.0083	0.0103	0.0988	0.1506	0.1718	0.0167	0.0750	0.0212	0.4463	0.0003
1	0.0034	0.0044	0.0632	0.1580	0.4279	0.0240	0.2884	0.0118	0.0144	0.0045
2	0.0057	0.0087	0.0641	0.1094	0.5057	0.0141	0.2659	0.0143	0.0111	0.0011
3	0.0062	0.0101	0.0627	0.1301	0.5491	0.0118	0.1968	0.0154	0.0159	0.0017
4	0.0018	0.0036	0.0339	0.0843	0.6023	0.0113	0.2427	0.0079	0.0109	0.0013
5	0.0044	0.0080	0.0827	0.1667	0.5055	0.0165	0.1213	0.0149	0.0774	0.0027
6	0.0034	0.0051	0.0605	0.1625	0.4867	0.0242	0.2271	0.0105	0.0140	0.0061

In stratified random sampling, each stratum is formed by 10 different classes following a frequency-based distribution. It means the subsamples are randomly made from each attack class, as well as Normal records, maintaining the same proportion as in the overall population. The training subset is then scaled using the Min-Max scaler, and informative features are selected using the Elastic Net and SFS feature selection algorithms, choosing 24, 24, and 8 features for BB, XGBoost, and RF, respectively.

Each classifier generates a matrix consisting of probability scores. The probability score of an input sample for BB is calculated as the mean of the class probabilities of trees. The XGBoost algorithm computes the sum of outputs from the leaves of each individual tree, also known as a margin score. This total is then converted into a range between 0 and 1, interpretable as a probability score, using the sigmoid function. The first seven rows of the matrix produced by BB are shown in Table 6, consisting of 10 columns representing 10 classes (9 attack plus Normal) and N rows, where N indicates the number of data samples in the validation subset.

To calculate the prediction errors, the validation set is input into the BB and XGBoost classifiers. Since the validation subset has 35,068 records, representing 20% of the training set, each model generates a probability matrix consisting of 35,068 rows. Both the probability matrix and the confusion matrix produced by the XGBoost, and BB classifiers are used as the inputs of Algorithm #1, presented in Figure 3. This algorithm processes the outputs from XGBoost and BB to compute the prediction errors caused by class overlap inherent to the dataset. Algorithm #1 identifies the smallest membership score errors made by BB and XGBoost, resulting from these classifiers incorrectly estimating the likelihood of the training samples belonging to a specific class type. The process calculates the mean and standard deviation of the errors, which are then utilized during testing to minimize membership errors.

The output of Algorithm #1 is structured as a nested hashmap with elements holding unordered lists, constituting Level-1 as shown in Figure 4, and consisting of 10 items representing all attack classes along with the Normal class. Each item in Level-1 points to 9 sub-items in Level-2, with each sub-item not containing the item itself from level-1. For each sub-item in Level-2 of the hashmap, there is a corresponding two-element list in Level-3. For further clarification, this hashmap can be visualized as two, two-dimensional arrays- one storing mean values and another standard deviation values- with the same dimensionality as the confusion matrix (10 by 10). Each row and column represent nine different attack classes along with the Normal class with the same order, similar to the rows and columns of the confusion matrix. These arrays store zeros along their main diagonal. The reason is that the aim of Algorithm #1 is to compute the prediction errors or those errors existing in the membership scores. Since the main diagonal, which holds true positives in a confusion matrix, contains no error to calculate, and thus these entries are not represented in the hashmap as well.

Algorithm #1 employs the confusion matrix to calculate differences: if a member of class A is misclassified as a member of class B for n different times, then this algorithm iterates n times through these instances to calculate the difference between the probability scores of class B and class A, as well as class B and the other eight classes. If the difference between class B and class A is smaller than those of the other eight differences, this value is stored in a temporary array (DL) for further processing. Otherwise, the value is discarded. In the next step, the mean and standard deviation of the stored values are calculated and kept in arrays OLM and OLSD, respectively. These mean and standard deviation values are placed at the third level of the hashmap, representing the outputs of Algorithm #1, where class A is an element of the first level and class B is an element of the second level.

To illustrate how Algorithm #1 works, we provide a step-by-step trace of its application on the Analysis and the Backdoor attack classes. The first row of Table 7 (a) represents a confusion matrix for the Analysis attack, and Table 7 (b) represents the probability scores generated by the BB classifier in a two-dimensional array or matrix form after it is trained and its performance evaluated on the validation subset. Values of these matrices are held by CM and PS, two-dimensional array variables in the pseudocode of Algorithm #1, respectively. Initially, DL, OLM, and OLSD are empty lists that eventually hold values for distance, mean (μ), and standard deviation (σ), respectively. AL is another list containing the Normal and all nine attack classes in an arbitrary order.

Next, consider the initialization steps in lines 1 through line 3 for the very first iteration of this algorithm:

Algorithm #1 – Compute Means and Standard Deviations

```

Mean-Standard-Deviation(CM, PS)
in:
    two-dimensional array  $CM_{10 \times 10}$  holding the confusion matrix
    two-dimensional array  $PS_{n \times 10}$  holding the membership scores with  $n$  = the number of
    samples of validation subset
out:
    two-dimensional array  $OLM_{10 \times 10}$  initialized with zero
    two-dimensional array  $OLSD_{10 \times 10}$  initialized with zero
local:
    empty array DL representing the minimum value of the membership scores difference
    empty variable  $\sigma$  representing the computed Standard Deviation value
    empty variable  $\mu$  representing the computed Mean value
    empty variables DT and D hold the values obtained by subtracting the membership scores
constant:
    array AL = {Analysis, Backdoor, DoS, Exploits, Fuzzers, Generic, Normal, Recon, Shellcode, Worms}

1  for all  $x \in AL$  do                                ▷ AL is a list
2      for all  $y \in (AL - x)$  do
3          if  $CM_{x,y} > 0$  then
4               $j \leftarrow 0$ 
5              for  $i \leftarrow 1 \dots CM_{x,y}$  do
6                   $D \leftarrow PS_{i,y} - PS_{i,x}$                 ▷ where x is misclassified as y
7                  for all  $z \in (AL - \{x, y\})$  do
8                       $DT \leftarrow PS_{i,y} - PS_{i,z}$ 
9                      if  $DT < D$  then
10                         count  $\leftarrow 1$ 
11                     end if
12                 end for
13                 if count = 0 then
14                      $DL_j \leftarrow D$ 
15                      $j \leftarrow j + 1$ 
16                 end if
17             end for
18             sum  $\leftarrow 0$  and  $N \leftarrow \text{length}(DL)$ 
19             for  $h \leftarrow 1 \dots \text{length}(DL)$  do
20                 sum  $\leftarrow \text{sum} + DL_h$ 
21             end for
22              $\mu \leftarrow \frac{\text{sum}}{N}$                                 ▷ where N indicates the number of elements in DL
23              $\sigma \leftarrow \sqrt{\frac{1}{N} \sum_{i=1}^N (DL_i - \mu)^2}$ 
24              $OLM_{x,y} \leftarrow \mu$ 
25              $OLSD_{x,y} \leftarrow \sigma$ 
26         end if
27     end for
28 end for
29 return OLM, OLSD

```

FIGURE 3 Pseudocode for Algorithm #1.

- The Analysis class is removed from the class list held in AL and placed in variable x as in line 1.
- In line 2, the Backdoor attack class is placed in variable y.
- Starting in line 3 and to compute the elements of both mean and standard deviation arrays, Algorithm #1 counts the false negatives for which Analysis is the target class while Backdoor is mistaken as Analysis due to misclassification. For this example, 124 records of Analysis are misclassified as Backdoor, Table 7 (a). Since the false-negative count is greater than zero for the Backdoor attack class, the difference between its prediction scores and those of Analysis are calculated for the corresponding data points. For instance, in the case of the Analysis and Backdoor attack classes, we have 124 rows and 10 columns of probability scores generated by the BB classifier.

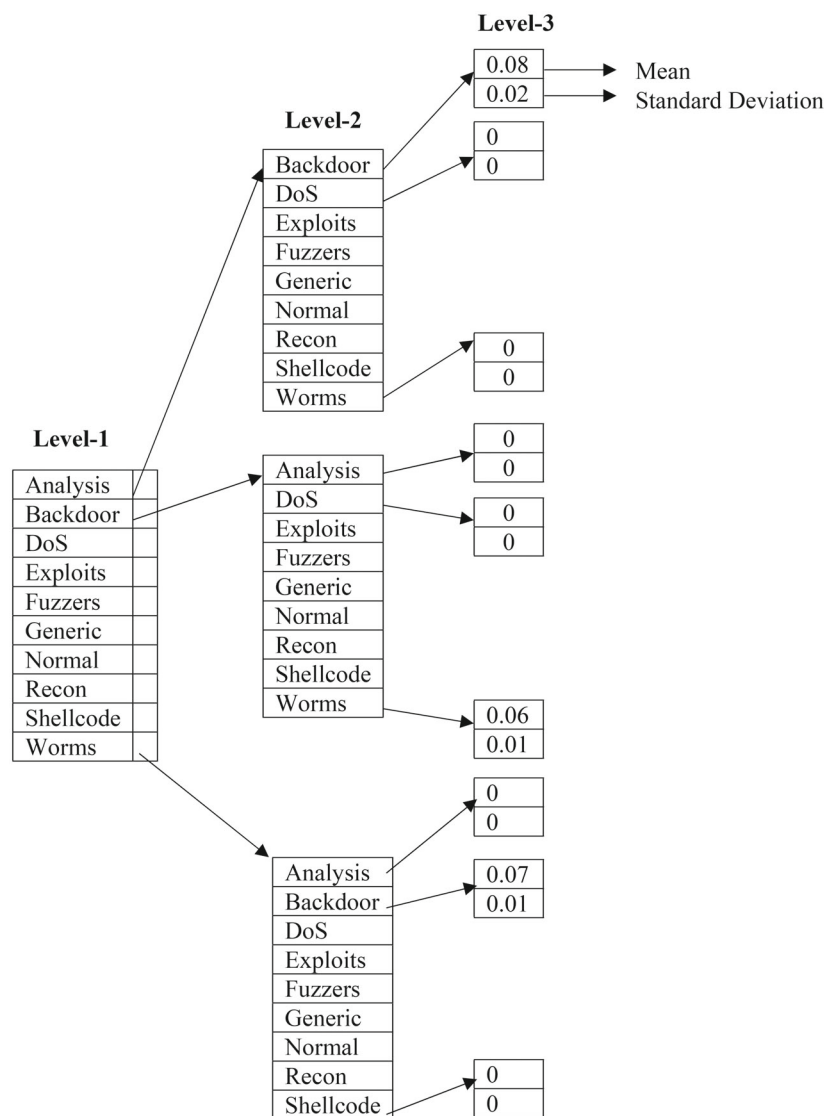


FIGURE 4 Illustration of partial output by Algorithm #1.

Execution now reaches the for-loop in line 5 during the first iteration.

- In line 5, the algorithm starts with the first row of the probability score matrix, where the likelihood of a new Analysis attack sample being classified as Analysis, Backdoor, DoS, Exploits, Fuzzers, Generic, Normal, Reconnaissance, Shellcode, and Worms is 0.78, 0.81, 0.02, 0.01, 0.24, 0.08, 0.11, 0.19, 0.08, 0.22, respectively, Table 7 (b).
- In line 6, the differences between the Backdoor score and the scores of other attack classes are calculated.
- In line 6, since the difference between the score for Analysis (the target class with a score of 0.78) and the score for Backdoor (incorrectly predicted class with a score of 0.81) is less than other differences, that is, $(0.81 - 0.78 = 0.03) < (0.81 - 0.02 = 0.79)$ and so on, the value 0.03 is stored in the distance variable D.
- The difference value stored in D is transferred to the array DL in line 14.

The for-loop in line 5 repeats 123 times. In the second iteration, the algorithm accesses the second row of the probability scores matrix with the values of 0.09, 0.54, 0.01, 0.03, 0.41, 0.04, 0.01, 0.06, 0.12 and 0.14, Table 7 (b).

- Line 6, calculates the differences of the scores.

TABLE 7 (a) Confusion matrix and (b) First two rows of probability scores matrix.

(a)										
	Analysis	Backdoor	DoS	Exploits	Fuzzers	Generic	Normal	Recon	Shellcode	Worms
Analysis	501	124	0	0	46	0	3	3	0	0
Backdoor	0	302	0	0	0	12	0	0	0	1
DoS	3	0	270	4	0	0	0	0	5	0
Exploits	0	0	0	254	8	0	50	0	0	0
Fuzzers	3	0	0	32	345	0	23	0	0	0
Generic	1	0	0	0	9	138	0	0	0	0
Normal	0	0	0	54	78	0	876	0	0	0
Recon	0	0	0	0	0	0	8	132	0	0
Shellcode	0	0	0	17	0	0	0	1	187	0
Worms	1	0	0	1	0	0	0	0	2	74
(b)										
Index	Analysis	Backdoor	DoS	Exploits	Fuzzers	Generic	Normal	Recon	Shellcode	Worms
1	0.78	0.81	0.02	0.01	0.24	0.08	0.11	0.19	0.08	0.22
2	0.09	0.54	0.01	0.03	0.41	0.04	0.01	0.06	0.12	0.14

- Since the difference between the Backdoor score and the Analysis score ($0.54 - 0.09 = 0.45$) is not less than the difference between the Backdoor score and the Fuzzers score ($0.54 - 0.41 = 0.13$), the difference will not be stored in the distance variable D. This computation repeats for the other 122 data points.
- In lines 22 through 25, the mean and the standard deviation for all those values in the distance array DL are calculated and kept in the mean and standard deviation arrays, namely OLM and OLSD, respectively, for the Analysis and Backdoor elements. We now have the range of values for which the errors due to data overlap need to be minimized.

In the next step, we determine whether the range of values calculated for each attack class overlaps. If the ranges overlap, the classifier will have difficulty distinguishing between the two overlapping classes. Addressing the overlap will help minimize the confusion for the ensemble classifier when making predictions for the test dataset. If the calculated ranges of two classes overlap, the range of the class with the larger false-negative count will be kept and the range of a class with the smaller false-negative count will be changed to zero. For example, if the range of values for Backdoor and Analysis, and Backdoor and Worms overlap, and if Analysis data are misclassified as Backdoor for 124 data points and Worms data are misclassified as Backdoor for 2 data points, the mean and standard deviation calculated for Backdoor and Analysis will be kept in the list, while the mean and standard deviation will be changed to zero for the case where Worms attack records are misclassified as Backdoor.

In other words, assume x has a value in the range of $[\mu_x - \sigma_x, \mu_x + \sigma_x]$ where the records in class A are misclassified as class B, and y assumes a value in the range of $[\mu_y - \sigma_y, \mu_y + \sigma_y]$ where the records in class C are incorrectly identified as class B; if the ranges for x and y overlap:

$$\begin{aligned}
 &\text{If } \mu_y \in [\mu_x - \sigma_x, \mu_x + \sigma_x] \\
 &\text{Else if } (\mu_y + \sigma_y) \leq (\mu_x + \sigma_x) \text{ and } (\mu_y - \sigma_y) \geq (\mu_x - \sigma_x) \\
 &\text{Else if } (\mu_y - \sigma_y) \geq (\mu_x - \sigma_x) \text{ and } (\mu_y + \sigma_y) \leq (\mu_x + \sigma_x)
 \end{aligned}$$

In this case, the values of mean and standard deviation for y will remain unchanged if and only if the number of records in class A that are incorrectly classified as class B is greater than the number of records in class C that are misidentified as class B and then the values of mean and standard deviation for x will be changed to zero.

Figure 5 shows that the Fuzzers, Backdoor, and Normal classes overlap. In this figure, Fuzzers is associated with a range of values between $0.09 - 0.02 = 0.07$ and $0.09 + 0.02 = 0.11$; Backdoor has a range of values between $0.08 - 0.02 = 0.06$ and $0.08 + 0.02 = 0.1$; and Normal is associated with a range of values between $0.04 - 0.03 = 0.01$ and $0.04 + 0.03 = 0.07$, where Analysis is incorrectly predicted as Fuzzers, Backdoor, and Normal, respectively. These values are taken from Table 8 (b). After identifying the overlaps, Table 8 (c) shows the final mean and standard deviation values

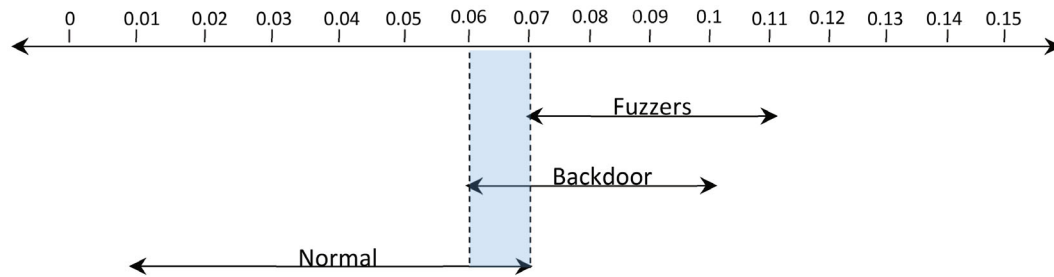


FIGURE 5 An illustration for the prediction error ranges for three classes for Algorithm #1.

TABLE 8 (a) The confusion matrix, (b) Mean and standard deviation (SD) arrays through Algorithm #1 and (c) Mean and standard deviation (SD) arrays after the range revision.

	Analysis	Backdoor	DoS	Exploits	Fuzzers	Generic	Normal	Recon	Shellcode	Worms
(a)										
Analysis	501	124	0	0	46	0	3	3	0	0
(b)										
Analysis	Mean	0.08	0	0	0.09	0	0.04	1.30	0	0
	SD	0.02	0	0	0.02	0	0.03	0.10	0	0
(c)										
Analysis	Mean	0.08	0	0	0	0	0	1.30	0	0
	SD	0.02	0	0	0	0	0	0.10	0	0

for all classes. In Table 8 (c), the mean and standard deviation values for the Recon class remain unchanged since its error range, from $1.3 - 0.1 = 1.2$ to $1.3 + 0.1 = 1.4$, does not overlap with other ranges. Since the ranges for Fuzzers, Backdoor, and Normal overlap, and more Analysis records are misclassified as Backdoor, the mean and standard deviation values calculated for Backdoor remain unchanged, while those for Fuzzers and Normal are set to zero.

The process of finding and addressing range overlaps repeats for all 10 classes. This procedure generates a list of mean and standard deviation values, as shown in Figure 4, Which is used in the test phase to minimize prediction errors resulting from class overlap.

4.2 | Testing

In this phase, the mean and standard deviation values computed by Algorithm #1 are used to reduce the errors made by the XGBoost and BB classifiers in identifying unseen data for class membership. Due to class data overlap within the UNSW-NB15 dataset, new samples mimicking the behavior of data points from another attack class are most likely to be misclassified. The probability scores matrix generated by the classifiers for a new sample in class A contains an error if this sample is incorrectly classified as class B. In this case, the probability matrix score for class A is lower than that for class B, whereas it should be the opposite. The difference between the probability scores for classes A and B is considered the error value. To mitigate this error, we apply Algorithm #2, as outlined in Figure 6, on the probability score matrices generated for the XGBoost and BB classifiers.

The test set is utilized to evaluate the performance of the proposed design, and therefore, the classifier does not know the value of the real target variable and errors cannot be calculated using the ground truth. Algorithm #2 is designed to go through the membership scores generated by the XGBoost and BB classifiers during the training and validation phases in order to calculate the errors for the test subset regardless of the values of the real target variables. The functionality of Algorithm #2 is elaborated in the following steps:

- In lines 3 to 8, the maximum membership score from the 10 different scores generated by either the XGBoost or BB classifier for each testing sample is identified. This maximum score is stored in the temporary variable *max* and its index value is kept in a temporary variable *index_max*.

Algorithm #2 Membership Score Modification

Membership-Score-Modification (PS, OLM, OLSD)

in:

two-dimensional array $PS_{n \times 10}$ holding the membership scores, n = the number of samples in the test subset

two-dimensional array $OLM_{10 \times 10}$

two-dimensional array $OLSD_{10 \times 10}$

out:

two-dimensional array $PS_{n \times 10}$ holding the (modified) membership scores, n = the number of samples in the test subset

```

1: for  $i \leftarrow 1 \dots n$  do
2:    $max \leftarrow 0$   $\triangleright$  holding zero in max to find the maximum membership score from line 3 to 8
3:   for  $j \leftarrow 1 \dots 10$  do
4:     if  $PS_{i,j} > max$  then
5:        $max \leftarrow PS_{i,j}$ 
6:        $index\_max \leftarrow j$ 
7:     end if
8:   end for
9:    $min \leftarrow 10^{10}$   $\triangleright$  holding a very big constant value in min to find the minimum values obtained thru lines 10 to 15
10:  for  $j \leftarrow 1 \dots 10$  do
11:    if  $((max - PS_{i,j}) < min)$  and  $(index\_max \neq j)$  then
12:       $min \leftarrow max - PS_{i,j}$ 
13:       $index\_min \leftarrow j$ 
14:    end if
15:  end for
16:  if  $(min \geq (OLM_{index\_min, index\_max} - OLSD_{index\_min, index\_max}))$  and
     $(min \leq (OLM_{index\_min, index\_max} + OLSD_{index\_min, index\_max}))$  then
17:     $PS_{i, index\_min} \leftarrow (PS_{i, index\_min} + OLM_{index\_min, index\_max})$ 
18:  end if
19: end for
20: return PS

```

FIGURE 6 Pseudocode for Algorithm #2.

- The algorithm calculates the difference between max and the nine other membership scores from lines 10 through 15. It computes these differences using the values from the probability score matrix across the columns, with max representing the maximum class membership score computed earlier. It picks the smallest such difference to store in variables min and $index_min$, which represent the difference value and class membership, respectively. These lines represent the computations that determine the membership score corresponding to a class which is the second most likely.
- In lines 16 and 17, the corresponding probability score matrix value is adjusted using the mean value associated with the class indices $index_min$ and $index_max$, which are referred to in the matrices OLM and OLSD. These indices correspond to the attack classes with the same ordering as in the list AL; for instance, $index_min = 1$ means Analysis. The operation in Line 17 increases the likelihood of a sample being correctly classified by adding the corresponding mean value calculated by Algorithm #1.

Using the mean and standard deviation values calculated by Algorithms #1 and #2 effectively mitigates errors in the probability matrices where errors arise due to class overlap. It should be noted that although errors are not entirely eliminated with this algorithm, they are significantly reduced.

The modified membership scores, along with those obtained by the augmented RF during the testing phase, are used to make the final prediction through a majority vote. Since hard voting is utilized in this method, the membership scores measured by each model for a particular class label are summed and the class with the highest total score is predicted in the majority vote. The overall classifier development steps are presented in Figure 7.

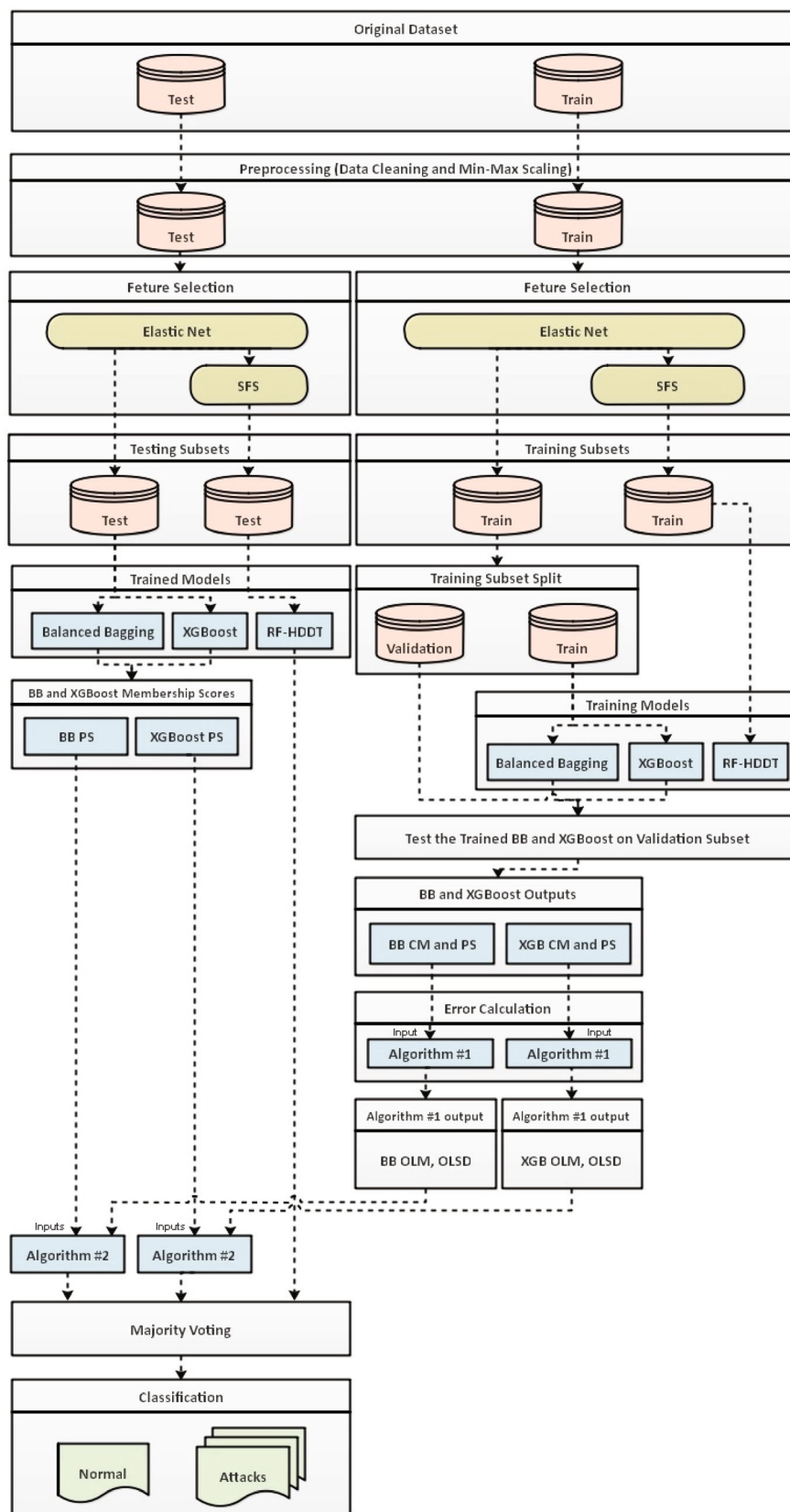


FIGURE 7 Classifier development process schematic diagram.

5 | SIMULATION RESULTS

In this section, we implement the proposed ensemble classifier and evaluate its performance on the UNSW-NB15 dataset for both binary and multi-category classification. The results are then compared with those published by other researchers for the same dataset. We profile the performance of each machine learning classifier using key metrics including sensitivity, specificity, false positive rate, false negative rate, precision, and f-measure for a comprehensive comparison.

5.1 | Binary Classification

The performance of the ensemble classifier on the test set with binary class labels, attack versus Normal, is presented in Table 9, showing that 790 out of 45,332 attack records are misclassified. This corresponds to a missed alarm rate of 1.7%, indicating that less than 2% of the attacks are misclassified as non-attack or normal, with the sensitivity for the attack class being 98.26%. On the other hand, 1022 of 37,000 Normal records are incorrectly classified as attack, reflecting a false alarm rate of less than 3%. Several evaluation metric values are included in Table 9 in order to comprehensively assess the performance of the proposed classifier design.

The main objective of any intrusion detection system (IDS) is to identify patterns of network traffic that may imply a suspicious activity. Accordingly, the performance of the proposed IDS on UNSW-NB15 data is competitive in comparison with the other studies reported in the literature as shown in Table 11. The second column from the left in Table 10 presents performance metrics associated with the attack class, suggesting that the proposed classifier design outperforms all those listed in Table 11, specifically studies reported in Refs. 15, 49–53.

The authors in Ref. 15 implemented MLP as an anomaly detection system for binary classification. They employed the Recursive Feature Elimination algorithm along with the RF classifier for dimensionality reduction, selecting the top four informative features. The original training and testing subsets proposed by Ref. 29, were used for training and evaluating the model. The MLP-based IDS achieved a sensitivity of 85% and an accuracy of 89% on the test subset of UNSW-NB15: 15% of the attack traces and 2% of the normal records were misclassified.

Bayu et al.⁴⁸ applied Gradient Boosted Machine (GBM) to three datasets, including UNSW-NB15, without implementing any feature selection technique, retaining all 47 features for the training and testing phases. The results showed that GBM outperformed four other algorithms, RF, Deep

TABLE 9 Confusion matrix for test data for binary classification.

Actual	Predicted	
	Normal	Attack
Normal	35,978	1022
Attack	790	44,542

TABLE 10 Performance of the proposed design for binary classification.

Class	Metrics					
	Sensitivity (%)	Specificity (%)	False positive rate	False negative rate	Precision (%)	F-measure (%)
Normal	97.24	98.26	0.017	0.028	97.85	97.75
Attack	98.26	97.24	0.028	0.017	97.76	98.01

TABLE 11 Comparison of the proposed classifier design with four others cited (NR: Not Reported).

Metrics	Proposed design	15	49	50	51	52	53
Sensitivity (%)	98.26	85.00	NR	98.47	91.30	92.10	99.74
FNR (%)	01.74	15.00	NR	NR	NR	08.10	1.31
FPR (%)	02.76	02.00	02.97	02.18	08.90	05.80	NR
Precision (%)	97.76	99.00	NR	NR	91.60	94.10	91.66
F-measure (%)	98.01	91.00	NR	NR	NR	93.08	95.53
Accuracy (%)	97.80	89.00	95.08	94.11	91.27	92.20	98.82

Neural Network (DNN), SVM and CART, with an average accuracy of 93.64% and missed alarm rate of 0.0206 across the NSL-KDD, UNSW-NB15 and GPRS datasets. While running GBM on UNSW-NB15 alone provided an accuracy of 95.08% and a false alarm rate of 2.97% using 10-fold cross-validation and the accuracy of 91.31% and false alarm rate of 8.60% using the Hold-Out method on the original UNSW-NB15 training and testing subsets.

In Ref. 49, nominal features were converted to numerical, and then the Min-Max normalization method was utilized to scale down the values to the range of 0 to 1. They used 5-fold cross-validation without resampling to generate the test and training subsets, calculating average values for sensitivity, false alarm rate, and accuracy across the folds. The authors utilized SVM by taking advantage of hyper clique property of hypergraph to improve SVM performance. This optimization also included feature selection. The SVM algorithm was trained with the entire 47 features and then the results were compared with the case when SVM was trained with the optimal number of feature subsets. The optimal feature subset is not reported. However, the number of optimal features is in the range of 30 to 35. The feature selection significantly influenced the proposed model, delivering a sensitivity of 98.47% and a false alarm rate of 2.18%. However, an accuracy of 94.11% and a false alarm rate of 2.18% suggest a relatively high missed alarm rate, which is not reported.

In Ref. 50 a hybrid feature selector consisting of particle swarm optimization, ant-colony algorithm, and genetic algorithm was used to reduce the dimensionality of the training dataset. The performance of the reduced error pruning tree (REPT) was evaluated to determine the most informative features during the feature selection phase. A two-level ensemble-based classifier comprising a rotation forest and a bagging classifier was implemented on the NSL-KDD and the UNSW-NB15 datasets. The classifier achieved a sensitivity of 91.30% while correctly classifying 91.60% of the positive samples.

Researchers in Ref. 51 proposed a parallel K-medoids algorithm using map reduction functions to cluster the instances. In this study, 8.10% of the malicious activities were misclassified, while 5.8% of the normal class members were classified as belonging to attack classes.

Zhang, et al.⁵³ proposed a NIDS method consisting of a data preprocessing module, an imbalance processing module, and a classification decision module, namely SGM-CNN. The performance of the proposed model was evaluated on the UNSW-NB15 and CICIDS2017 datasets in terms of binary classification and multi-category classification. Their design achieved a 99.74% detection rate as a binary classifier.

5.2 | Multiclass Classification

In this section, the performance evaluation of the ensemble classifier, without the application of the proposed Algorithms #1 and #2, for the multi-class case is shown in Table 12 in terms of a confusion matrix. Although the ensemble design has effectively improved the performance of each base classifier, it still produces a relatively large number of false negatives. To address this issue, Algorithms #1 and #2 were developed, culminating in what is referred to as the proposed ensemble classifier or design.

The performance of the proposed ensemble classifier, with the Algorithms #1 and #2 integrated, is shown in Table 12. Comparing results in Table 12 with those in Table 13 indicates a significant improvement for most classes, including the Normal class which notably reduce in the number of false negatives. This enhancement validates the effectiveness and utility of Algorithms #1 and #2.

According to Table 13, the number of attack records misclassified as Normal has significantly decreased compared to the same column in Table 12, indicating a substantial improvement in the overall detection rate of attack class records. Furthermore, a comparison of the diagonals in

TABLE 12 Confusion matrix showing the performance of the ensemble classifier.

	Analysis	Backdoor	DoS	Exploits	Fuzzers	Generic	Normal	Recon	Shellcode	Worms
Analysis	327	193	4	1	18	0	130	0	4	0
Backdoor	298	154	8	8	23	0	76	1	15	0
DoS	1477	737	775	435	221	0	228	45	164	7
Exploits	1322	1583	51	6197	257	0	667	490	420	145
Fuzzers	651	386	8	14	1837	0	2604	7	540	15
Generic	15	30	37	390	99	18,145	63	7	72	13
Normal	842	1	3	28	1427	0	34,545	0	154	0
Recon	97	210	0	35	14	0	31	2967	128	14
Shellcode	11	0	0	0	7	0	9	3	347	1
Worms	0	0	0	1	0	0	1	0	3	39

TABLE 13 Confusion matrix showing the performance of the proposed design.

	Analysis	Backdoor	DoS	Exploits	Fuzzers	Generic	Normal	Recon	Shellcode	Worms
Analysis	327	193	4	1	18	0	130	0	4	0
Backdoor	126	405	0	0	13	0	39	0	0	0
DoS	1604	625	1110	228	92	0	178	59	164	29
Exploits	779	830	39	8511	57	0	338	221	213	144
Fuzzers	482	491	4	34	4380	0	0	10	646	15
Generic	15	30	37	390	99	18,145	63	7	72	13
Normal	200	0	0	0	668	0	35,978	0	154	0
Recon	103	207	0	35	14	0	31	2967	126	13
Shellcode	0	0	0	0	7	0	9	3	358	1
Worms	0	0	0	1	0	0	2	0	5	36

TABLE 14 Performance of the proposed design for the multi-class case.

Class	Metric						
	Accuracy	Sensitivity	Specificity	FPR	FNR	Precision	F-measure
Analysis	95.56	48.30	95.95	0.041	0.51	0.15	64.25
Backdoor	96.89	69.47	97.09	0.029	0.31	0.84	80.99
Dos	96.27	27.15	99.89	0.001	0.73	0.42	42.70
Exploits	95.98	76.46	99.03	0.009	0.24	0.84	86.29
Fuzzers	96.78	72.25	98.73	0.012	0.28	0.77	83.44
Generic	99.12	96.15	100.00	0.000	0.28	1.00	98.04
Normal	97.81	97.24	98.26	0.017	0.02	0.97	97.75
Reconnaissance	95.39	84.87	95.86	0.041	0.15	0.61	90.03
Shellcode	99.31	94.71	98.33	0.017	0.05	0.34	96.49
Worms	99.73	81.82	99.74	0.003	0.18	0.24	89.90

Tables 12 and 13 reveals enhanced detection rates for most attack classes. However, the presence of many nonzero off-diagonal entries in the confusion matrix of Table 13 suggests that the classifier still struggles to distinguish among certain attack classes for a significant number of attack records.

Table 14 presents the performance of the proposed ensemble classifier in terms of benchmark metrics. As shown in Tables 13 and 14, the proportion of attack records incorrectly categorized as Normal is 4.14% out of an overall missed alarm rate of 28%. The remaining 23.86% of the missed alarm rate is attributed to misclassifications among attack classes, which may not be as problematic for intrusion detection purposes since it indicates that an attack was detected but the specific type of attack was not identified correctly. Although 4.14% missed alarm rate for attack records alone could be detrimental to an intrusion detection system, our design achieves better results compared to those in other studies as shown in Table 14.

Normal traces are also misclassified as Shellcode, Fuzzers and Analysis, suggesting a false alarm rate of approximately 3% and a sensitivity of 97.24% as shown in Table 14. This attributed to these attack types mimicking the behavior of Normal records,^{32,54,55} leading to some attacks being incorrectly traced and predicted as Normal activity. In the associated confusion matrix in Table 13, 19.20% of Analysis, 6.69% of Backdoor, 4.35% of DoS, 3.04% of Exploits, 0.33% of Generic, 0.88% of Reconnaissance, 2.38% of Shellcode, and 4.55% of Worms attack records are confused with Normal records.

A comparison of the proposed model with those reported in the literature is detailed in Tables 15 and 16. Many of the relevant performance metrics, including the missed alarm rate, which is critical for the intrusion detection context, are not reported in these studies. Therefore, the comparison is primarily based on accuracy and sensitivity, the only metrics commonly reported.

TABLE 15 Accuracy of the proposed design versus the accuracy of different classifier (NR: Not Reported).

Class	Accuracy (Proposed)	Accuracy ⁵⁶	Accuracy ⁵⁷	Accuracy ⁵⁸	Difference
Analysis	95.56	99.26	99.30	NR	−3.74
Backdoor	96.89	99.11	97.93	NR	−2.22
Dos	96.27	94.90	95.71	94.52	+0.56
Exploits	95.98	90.12	93.58	89.72	+2.40
Fuzzers	96.78	91.47	95.04	NR	+1.74
Generic	99.12	98.23	98.70	87.70	+0.42
Normal	97.81	93.54	94.59	98.64	−0.30
Reconnaissance	95.39	95.33	96.18	99.10	−3.71
Shellcode	99.31	99.40	98.33	NR	−0.09
Worms	99.73	99.92	99.78	NR	−0.14

Note: The bold values in Difference column represent how much the other models in the highlighted cells, worked better than the proposed model. Highlighted cells indicate the models that performed best.

TABLE 16 Sensitivity of the proposed model versus the sensitivity of different classifiers (NR: Not Reported).

Attack Type	Sensitivity (Proposed)	Sensitivity ⁵⁷	Sensitivity ⁵⁸	Sensitivity ⁵⁹	Difference
Analysis	48.30	20.45	NR	NR	+27.85
Backdoor	69.47	67.32	NR	NR	+02.15
Dos	27.15	14.29	5.0	29.46	+12.86
Exploits	76.46	76.22	54.64	85.69	+00.24
Fuzzers	72.25	64.42	NR	NR	+07.83
Generic	96.15	81.37	96.72	98.58	−00.57
Normal	97.24	97.39	98.00	96.19	−00.76
Reconnaissance	84.87	46.04	71.70	79.42	+13.17
Shellcode	94.71	36.39	NR	NR	+58.32
Worms	81.82	18.37	NR	NR	+63.45

Note: The bold values in the last column show the extent to which the other models outperformed the proposed model. Highlighted cells denote the models that achieved the best performance.

Kumar et al.⁵⁹ developed the Unified Intrusion Detection System (UIDS) by generating the new training and test subsets from UNSW-NB15. They utilized the k-means clustering algorithm to increase the attack sensitivity as the k-means clustering algorithm was able to identify the similarities between different attack classes. In each cluster, the number of records in some type of attack classes was more than the rest. The authors randomly picked 65% of the records of the dominating class categories to form a training dataset. The remaining 35% of the instances were used to build the test set. They also used information gain algorithm for feature selection, identifying 13 out of 47 features which improved the accuracy scores using C5.0, Chi-Squared Automatic Inference Detection (CHAID), Classification and Regression Tree (CART) is also known as Decision Tree (DT) and Quick Unbiased Efficient Statistical Tree (QUEST) algorithms. These algorithms were used to form the proposed UIDS model. Their study reported average sensitivity and F-measure values of 77.87% and 79.12%, respectively, with a 3.80% false alarm rate for Normal instances and an 86.15% attack sensitivity.

Papamartzivanos et al.⁵⁷ combined the decision tree and genetic algorithm to generate classification rules in their model called Dendron. The wrapper technique⁶⁰ was used for feature selection, resulted in 23 informative features being selected. The authors reported a sensitivity of 97.39% for the Normal class and an average false alarm rate of 2.61% as presented in Table 16. In their study, 10% of the instances for each of the 9 classes were used for building the training set, with the remaining 90% kept for the test set. To address the imbalance problem in multi-class classification within the UNSW-NB15 dataset, 50% of Worms attack class records were included in the training subset, with the remaining 50% kept to test the model. The proposed model in our study outperforms the others given the F-measure values, likely due to successful addressing of the imbalance and overlapping issues.

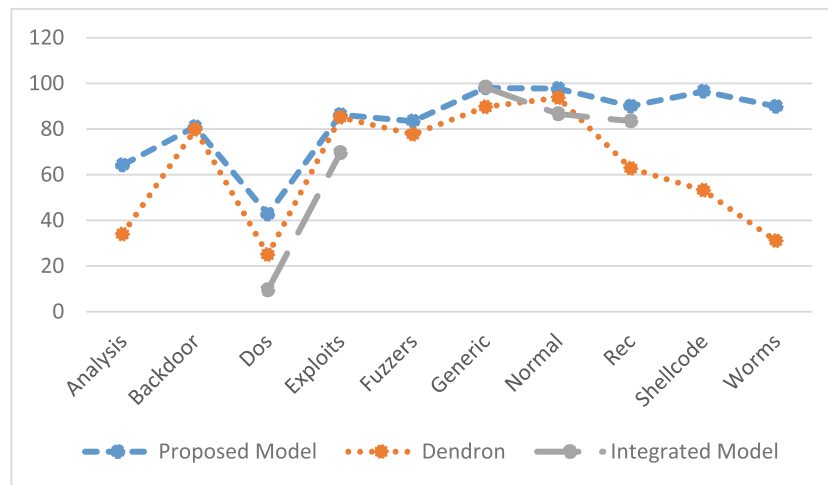


FIGURE 8 Comparison of the proposed design with two others in literature based on the F-measure.

The integrated rule-based classifier is trained to detect five class types to avoid overlapping.⁴⁸ The rules employed by this classifier, were generated from four tree-structured classification algorithms, namely C5.0, CHAID, CART, and QUEST. Training and test sets were built by excluding some instances from the original training and testing subsets using k-means clustering. These excluded instances belonged to Analysis, Backdoor, Fuzzers, Shellcode, and Worms attack types, which suffer from class overlap issues potentially causing poor classification performance. For the feature selection phase, the genetic algorithm was used and 22 features were selected. They reported good accuracy and sensitivity values for the Normal class. Also, the reconnaissance is detected successfully with an accuracy of 99.10% as shown in Table 15. However, the average accuracy and sensitivity values are 93.94% and 65.21%, respectively.

Figure 8 depicts the performance of the proposed ensemble design compared with two other models, Integrated⁵⁸ and Dendron,⁵⁷ in terms of the F-measure. As depicted, the proposed ensemble classifier outperforms both Dendron and Integrated for each class.

6 | CONCLUSIONS

In this research, we developed an ensemble classifier optimized for the UNSW-NB15 cybersecurity dataset, tailored for both binary and multi-category classification tasks. Our approach integrated comprehensive preprocessing techniques, robust tree-based classifiers, and a novel ensemble framework augmented with two newly devised algorithms. The efficacy of the ensemble model was benchmarked against existing models in the literature, applied to the same dataset.

For binary classification, our ensemble model accurately identified over 98% of attack instances, with less than 2% misclassified. The identification of Normal records was equally impressive, achieving a correct classification rate exceeding 97% and maintaining a false alarm rate under 3%. In the multi-class scenario, while there was a 28% misclassification rate for attack instances, only 4.14% were incorrectly labeled as Normal, with the remainder misclassified among other attack types. Notably, Normal records were classified with a high accuracy of 97.24%, and false alarms, where Normal activities were misinterpreted as attacks, occurred in just 2.76% of cases.

The focal point of this study was the effectiveness of the newly introduced Algorithms #1 and #2 in enhancing the performance of an ensemble classifier when faced with overlapping and imbalanced datasets. Future work will extend the application of these algorithms across various machine learning models and newly acquired IDS datasets, evaluating their performance against recent models and discrimination threshold detection techniques. Further refinements of the proposed algorithms will also aim to better address overlapping features and improve prediction scores.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are available in <https://cloudstor.aarnet.edu.au/plus/index.php/s/2DhnLGDdEEC> at <https://research.unsw.edu.au/projects/unsw-nb15-dataset>. These data were derived from the following resources available in the public domain: Moustafa, Nour, and Jill Slay, <https://research.unsw.edu.au/projects/unsw-nb15-dataset>.

ORCID

Zeinab Zoghi  <https://orcid.org/0000-0002-1285-0438>

REFERENCES

- Medha Pujari JN, Sun W, Jahan F, McCreary B, Niyaz Q, Devabhaktuni VK. *PortableVN: a generic Mobile application for network security testbeds*, in *proceedings of the international conference on security and management (SAM)*. The Steering Committee of the World Congress in Computer Science, Computer Engineering and Applied Computing (WorldComp); 2019:125-131.
- Zoghi Z, Serpen G. UNSW-NB15 computer security dataset: Analysis through visualization. *Secur Priv*. 2021;7(1):e331.
- Wang S, Yao X. Multiclass imbalance problems: analysis and potential solutions. *IEEE Trans Syst Man Cybern B Cybern*. 2012;42(4):1119-1130.
- Hajibabae P, Pourkamali-Anaraki F, Hariri-Ardebili MA. Kernel matrix approximation on class-imbalanced data with an application to scientific simulation. *IEEE Access*. 2021;9:83579-83591.
- Prati RC, Batista GE, Monard MC. Class imbalances versus class overlapping: an analysis of a learning system behavior. *Mexican International Conference on Artificial Intelligence*. Springer; 2004.
- Enayati S, Yang Z, Lu B, Vucetic S. A visualization approach for rapid labeling of clinical notes for smoking status extraction. *Proceedings of the Second Workshop on Data Science with Human in the Loop: Language Advances*; 2021.
- Błaszczyszki J, Stefanowski J. Actively balanced bagging for imbalanced data. *International Symposium on Methodologies for Intelligent Systems*. Springer; 2017.
- Hido S, Kashima H, Takahashi Y. Roughly balanced bagging for imbalanced data. *Stat Anal Data Min: ASA Data Sci J*. 2009;2(5-6):412-426.
- Chen T, Guestrin C. Xgboost: a scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*; 2016.
- Ho TK. Random decision forests. *Proceedings of 3rd International Conference on Document Analysis and Recognition*. IEEE; 1995.
- Smirti Dwibedi MP, Sun W. A comparative study on contemporary intrusion detection datasets for machine learning research. *2020 IEEE International Conference on Intelligence and Security Informatics (ISI)*. IEEE; 2020:1-6.
- Dlamini G, Fahim M. DGM: a data generative model to improve minority class presence in anomaly detection domain. *Neural Comput & Applic*. 2021;33:1-12.
- Kocher G, Kumar G. Analysis of machine learning algorithms with feature selection for intrusion detection using UNSW-NB15 dataset. *Available at SSRN*; 2021.
- Kilincer IF, Ertam F, Sengur A. Machine learning methods for cyber security intrusion detection: datasets and comparative study. *Comput Netw*. 2021;188:107840.
- Kanimozhi V, Jacob P. Unsw-nb15 dataset feature selection and network intrusion detection using deep learning. *Int J Recent Technol Eng*. 2019;7(5S2):443-446.
- Shults R, Levin E, Habibi R, et al. Capability of Matterport 3D camera for industrial archaeology sites inventory. *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*. 2019;42:1059-1064.
- HekmatiAthar S, Goins H, Samuel R, Byfield G, Anwar M. Data-driven forecasting of agitation for persons with dementia: a deep learning-based approach. *SN Comput Sci*. 2021;2(4):1-10.
- Heidari M, Jones JH. Using bert to extract topic-independent sentiment features for social media bot detection. *2020 11th IEEE Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*. IEEE; 2020.
- Heidari MAJ, James HJR, Uzuner O. An empirical study of machine learning algorithms for social media bot detection. *2021 IEEE International IOT, Electronics and Mechatronics Conference (IEMTRONICS)*. IEEE; 2021:1-5.
- Uzuner MHAJHJJaO. Deep contextualized word embedding for text-based online user profiling to detect social bots on twitter. *2020 International Conference on Data Mining Workshops (ICDMW), [ICDMW] 2020*. IEEE; 2020.
- Amirkhani F, Dashti A, Abedsoltan H, Mohammadi AH, Chau KW. Towards estimating absorption of major air pollutant gasses in ionic liquids using soft computing methods. *J Taiwan Inst Chem Eng*. 2021;127:109-118.
- Cieslak DA, Chawla NV. Learning decision trees for unbalanced data. *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer; 2008.
- Cieslak DA, Hoens TR, Chawla NV, Kegelmeyer WP. Hellinger distance decision trees are robust and skew-insensitive. *Data Min Knowl Disc*. 2012;24(1):136-158.
- Movassagh AA, Alzubi JA, Gheisari M, et al. Artificial neural networks training algorithm integrating invasive weed optimization with differential evolutionary model. *J Ambient Intell Humaniz Comput*. 2021;14:1-9.
- Alzubi OA, Alzubi JA, Alweshah M, Qiqieh I, Al-Shami S, Ramachandran M. An optimal pruning algorithm of classifier ensembles: dynamic programming approach. *Neural Comput & Applic*. 2020;32:16091-16107.
- Alzubi OA, Alzubi JA, Al-Zoubi AM, Hassonah MA, Kose U. An efficient malware detection approach with feature weighting based on Harris Hawks optimization. *Clust Comput*. 2022;25:1-19.
- Alzubi OA, Qiqieh I, Alzubi JA. Fusion of deep learning based cyberattack detection and classification model for intelligent systems. *Clust Comput*. 2023;26(2):1363-1374.
- Moustafa N. *Designing an online and reliable statistical anomaly detection framework for dealing with large high-speed network traffic*. University of New South Wales; 2017.
- Moustafa N, Slay J. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). *2015 Military Communications and Information Systems Conference (MilCIS)*. IEEE; 2015.
- Moustafa N. The UNSW-NB15 Dataset Description. 2018 June 3, 2020; Available from: <https://www.unsw.adfa.edu.au/unsw-canberra-cyber/cybersecurity/ADFA-NB15-Datasets/>
- Janarthanan T, Zargari S. Feature selection in UNSW-NB15 and KDDCUP'99 datasets. *2017 IEEE 26th International Symposium on Industrial Electronics (ISIE)*. IEEE; 2017.
- Moustafa N, Slay J. The evaluation of network anomaly detection systems: statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set. *Inf Secur J Glob Perspect*. 2016;25(1-3):18-31.
- Tabbakh SK, Habibi R, Vafadar S. Design and implementation of a framework based on augmented reality for phobia treatment applications. *2015 International Congress on Technology, Communication and Knowledge (ICTCK)*. IEEE; 2015.

34. Levin E, Roland W, Habibi R, An Z, Shults R. Rapid visual presentation to support geospatial big data processing. *Int Arch Photogram Rem Sens Spat Informat Sci*. 2020;43:463-470.
35. Levin E, Shults R, Habibi R, An Z, Roland W. Geospatial virtual reality for cyberlearning in the field of topographic surveying: moving towards a cost-effective mobile solution. *ISPRS Int J Geo Inf*. 2020;9(7):433.
36. Ixia. PerfectStorm. 1998 [cited June 3, 2020]; Available from: <https://www.ixiacom.com/products/perfectstorm>
37. Argus. [June 3, 2020]; Available from: <https://openargus.org/>
38. Zeek. [June 3, 2020]; Available from: <https://zeek.org/>
39. Fernández A, García S, del Jesus MJ, Herrera F. A study of the behaviour of linguistic fuzzy rule based classification systems in the framework of imbalanced data-sets. *Fuzzy Sets Syst*. 2008;159(18):2378-2398.
40. Liu J, Malekzadeh M, Mirian N, Song TA, Liu C, Dutta J. Artificial intelligence-based image enhancement in PET imaging: noise reduction and resolution enhancement. *PET Clinics*. 2021;16(4):553-576.
41. Al-Daweri MS, Zainol Ariffin KA, Abdullah S. An analysis of the KDD99 and UNSW-NB15 datasets for the intrusion detection system. *Symmetry*. 2020;12(10):1666.
42. Ahmad M, Riaz Q, Zeeshan M, Tahir H, Haider SA, Khan MS. Intrusion detection in internet of things using supervised machine learning based on application and transport layer features using UNSW-NB15 data-set. *EURASIP J Wirel Commun Netw*. 2021;2021(1):1-23.
43. Kasongo SM, Sun Y. Performance analysis of intrusion detection systems using a feature selection method on the UNSW-NB15 dataset. *J Big Data*. 2020;7(1):1-20.
44. Zhang J, Ling Y, Fu X, Yang X, Xiong G, Zhang R. Model of the intrusion detection system based on the integration of spatial-temporal features. *Comput Secur*. 2020;89:101681.
45. Kasongo SM, Sun Y. A deep learning method with wrapper based feature extraction for wireless intrusion detection system. *Comput Secur*. 2020;92:101752.
46. Barandela R, Valdovinos RM, Sánchez JS. New applications of ensembles of classifiers. *Pattern Anal Applic*. 2003;6(3):245-256.
47. Aridas C. Imbalanced-learn. 2019 [cited 2019]; Available from: <https://github.com/scikit-learn-contrib/imbalanced-learn/>
48. Nguyen TD, Shih MH, Srivastava D, Tirthapura S, Xu B. Stratified random sampling from streaming and stored data. *Distrib Parallel Databases*. 2020;39:1-46.
49. Tama BA, Rhee K-H. An in-depth experimental study of anomaly detection using gradient boosted machine. *Neural Comput & Applic*. 2019;31(4):955-965.
50. Gauthama Raman MR, Somu N, Jagarapu S, et al. An efficient intrusion detection technique based on support vector machine and improved binary gravitational search algorithm. *Artif Intell Rev*. 2020;53(5):3255-3286.
51. Tama BA, Comuzzi M, Rhee K-H. TSE-IDS: a two-stage classifier ensemble for intelligent anomaly-based intrusion detection system. *IEEE Access*. 2019;7:94497-94507.
52. Dahiya P, Srivastava DK. A comparative evolution of unsupervised techniques for effective network intrusion detection in hadoop. *International Conference on Advances in Computing and Data Sciences*. Springer; 2018.
53. Zhang H, Huang L, Wu CQ, Li Z. An effective convolutional neural network based on SMOTE and gaussian mixture model for intrusion detection in imbalanced dataset. *Comput Netw*. 2020;177:107315.
54. Kumar V, Sinha D, Das AK, Pandey SC, Goswami RT. An integrated rule based intrusion detection system: analysis on UNSW-NB15 data set and the real time online dataset. *Clust Comput*. 2019;23:1-22.
55. Yang Y, Zheng K, Wu C, Niu X, Yang Y. Building an effective intrusion detection system using the modified density peak clustering algorithm and deep belief networks. *Appl Sci*. 2019;9(2):238.
56. Sharma J, Giri C, Granmo OC, Goodwin M. Multi-layer intrusion detection system with ExtraTrees feature selection, extreme learning machine ensemble, and softmax aggregation. *EURASIP J Inf Secur*. 2019;2019(1):1-16.
57. Papamartzivanos D, Mármol FG, Kambourakis G. Dendron: genetic trees driven rule induction for network intrusion detection systems. *Futur Gener Comput Syst*. 2018;79:558-574.
58. Kumar V, Sinha D, Das AK, Pandey SC, Goswami RT. An integrated rule based intrusion detection system: analysis on UNSW-NB15 data set and the real time online dataset. *Clust Comput*. 2020;23(2):1397-1418.
59. Kumar V, Das AK, Sinha D. UIDS: a unified intrusion detection system for IoT environment. *Evol Intel*. 2021;14(1):47-59.
60. Kohavi R, John GH. Wrappers for feature subset selection. *Artif Intell*. 1997;97(1-2):273-324.

How to cite this article: Zoghi Z, Serpen G. Building an intrusion detection system on UNSW-NB15: Reducing the margin of error to deal with data overlap and imbalance. *Concurrency Computat Pract Exper*. 2024;36(25):e8242. doi: 10.1002/cpe.8242